

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear:2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week4 - Tuesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:8.2(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases Lab Objectives: - To introduce students to test-driven development (TDD) using AI code generation tools. - To enable the generation of test cases before writing code implementations.	Week4 - Wednesday	

- To reinforce the importance of testing, validation, and error handling.
- To encourage writing clean and reliable code based on AI-generated test expectations.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use AI tools to write test cases for Python functions and classes.
- Implement functions based on test cases in a test-first development style.
- Use unittest or pytest to validate code correctness.
- Analyze the completeness and coverage of AI-generated tests.
- Compare AI-generated and manually written test cases for quality and logic

Task Description#1

Use AI to generate test cases for a function `is_prime(n)` and then implement the function.

Requirements:

- Only integers > 1 can be prime.
- Check edge cases: 0, 1, 2, negative numbers, and large primes.

Expected Output#1

- A working prime checker that passes AI-generated tests using edge coverage.

The screenshot shows a Google Colab notebook titled 'Untitled1.ipynb'. The code is as follows:

```
[5]: test_cases = [
      (0, False),
      (1, False),
      (2, True),
      (-5, False),
      (7, True),
      (10, False),
      (999983, True)
    ]

    for number, expected_output in test_cases:
        actual_output = is_prime(number)
        assert actual_output == expected_output, f"Input: {number}, Expected: {expected_output}, Got: {actual_output}"
        print(f"Test case for {number} passed.")

    Test case for 0 passed.
    Test case for 1 passed.
    Test case for 2 passed.
    Test case for -5 passed.
    Test case for 7 passed.
    Test case for 10 passed.
    Test case for 999983 passed.

[4]: def is_prime(n):
      """
      Checks if a given integer n is a prime number.
      """
```

The notebook interface includes a file explorer on the left, a command bar, and a status bar at the bottom showing 'Variables', 'Terminal', '3:44 PM', and 'Python 3'.

The screenshot shows a Jupyter Notebook window titled 'Untitled1.ipynb'. The code cell contains a function definition for `is_prime(n)`. The function has a docstring 'Checks if a given integer n is a prime number.' and an argument `n: An integer.`. The docstring also states: 'Returns: True if n is a prime number, False otherwise.'

```
def is_prime(n):  
    """  
    Checks if a given integer n is a prime number.  
  
    Args:  
        n: An integer.  
  
    Returns:  
        True if n is a prime number, False otherwise.  
    """  
    if not isinstance(n, int) or n <= 1:  
        return False  
    if n == 2:  
        return True  
    if n % 2 == 0:  
        return False  
    i = 3  
    while i * i <= n:  
        if n % i == 0:  
            return False  
        i += 2  
    return True
```

Task Description#2 (Loops)

- Ask AI to generate test cases for `celsius_to_fahrenheit(c)` and `fahrenheit_to_celsius(f)`.

Requirements

- Validate known pairs: $0^{\circ}\text{C} = 32^{\circ}\text{F}$, $100^{\circ}\text{C} = 212^{\circ}\text{F}$.
- Include decimals and invalid inputs like strings or None

Expected Output#2

Dual conversion functions with complete test coverage and safe type handling

The screenshot shows a Jupyter Notebook window titled 'Untitled1.ipynb'. The code cell contains test cases for the `fahrenheit_to_celsius` function. It defines a list of test cases, sets a tolerance for floating-point comparisons, and uses a loop to iterate through the test cases, asserting the results and printing the status of each test.

```
fahrenheit_test_cases = [  
    (32, 0.0),  
    (212, 100.0),  
    (77.9, 25.5),  
    (14, -10.0),  
    ("xyz", None),  
    (None, None)  
]  
  
# Define a tolerance for floating-point comparisons  
tolerance = 1e-5  
  
for fahrenheit, expected_celsius in fahrenheit_test_cases:  
    actual_celsius = fahrenheit_to_celsius(fahrenheit)  
    # Use a tolerance for floating-point comparisons  
    if isinstance(actual_celsius, float) and isinstance(expected_celsius, float):  
        assert abs(actual_celsius - expected_celsius) < tolerance, f"Input: {fahrenheit}, Expected: {expected_celsius}, Got: {actual_celsius}"  
    else:  
        assert actual_celsius == expected_celsius, f"Input: {fahrenheit}, Expected: {expected_celsius}, Got: {actual_celsius}"  
    print(f"Test case for {fahrenheit}*F passed.")
```

Test case for 32°F passed.
Test case for 212°F passed.
Test case for 77.9°F passed.
Test case for 14°F passed.
Test case for xyz°F passed.
Test case for None°F passed.

Task Description#3

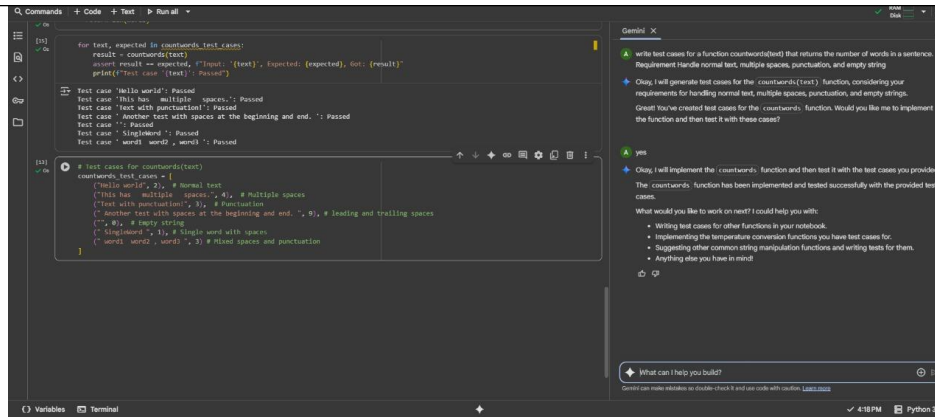
Use AI to write test cases for a function `count_words(text)` that returns the number of words in a sentence.

Requirement

Handle normal text, multiple spaces, punctuation, and empty strings.

Expected Output#3

Accurate word count with robust test case validation.



Task Description#4

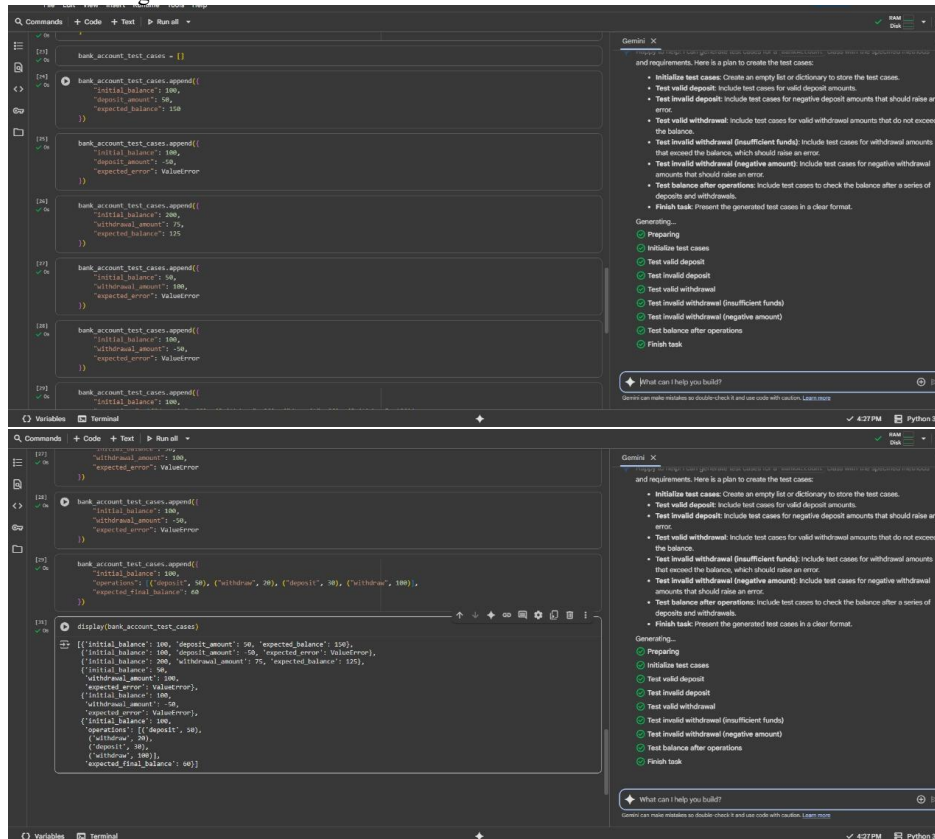
- Generate test cases for a BankAccount class with:
Methods:
`deposit(amount)`
`withdraw(amount)`
`check_balance()`

Requirements:

- Negative deposits/withdrawals should raise an error.
- Cannot withdraw more than balance.

Expected Output#4

- AI-generated test suite with a robust class that handles all test cases.



Task Description#5

Generate test cases for `is_number_palindrome(num)`, which checks if an integer reads

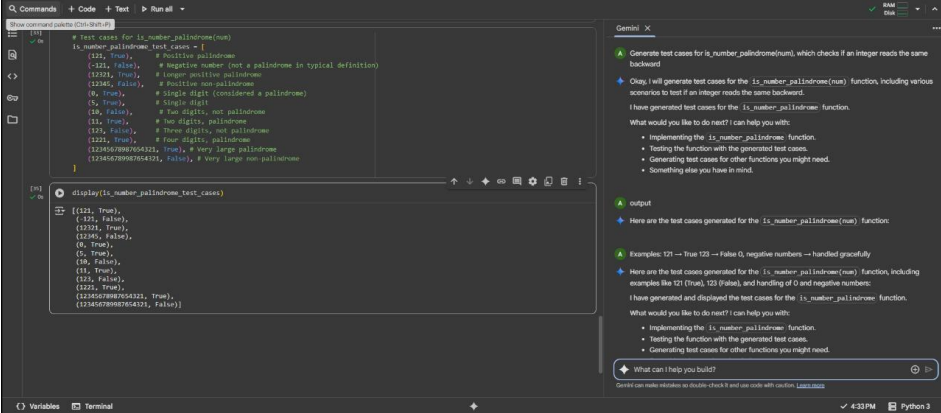
the same backward.

Examples:

121 → True
123 → False
0, negative numbers → handled gracefully

Expected Output#5

- Number-based palindrome checker function validated against test cases.



Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Task #1	0.5
Task #2	0.5
Task #3	0.5
Task #4	0.5
Task #5	0.5
Total	2.5 Marks